

Faculty of Computing

CS110: Fundamentals of Computer Programming

Class: BSAI 2026A

Lab 03: Arithmetic and Relational Operators

CLO 1	Understand the syntax and semantics of different programming constructs
CLO 3	Build a program and associated documentation using appropriate IDE and supplementary tools

Time: 2:00 pm - 4:50 pm

Instructor: Sahar Arshad
Lab Engineer: Urooj Akmal

Introduction:

This lab builds on Lab 02, where students practiced basic C++ program structure, variables, input/output, and debugging. The focus now shifts to using arithmetic operators to calculate values and relational operators to compare values. The activities are designed for beginners and move from short operator experiments to medium-complexity programs. No if/else statements or logical, unary, or bitwise operators are required in this lab.

Learning Objectives:

After completing this lab, you will be able to:

- Use the arithmetic operators +, -, *, /, and % correctly in C++ expressions.
- Explain integer division and the remainder produced by the % operator, and use them in small calculations.
- Use the relational operators ==, !=, >, <, >=, and <= and interpret their Boolean output as 1 (true) or 0 (false).
- Compile, test, and debug small programs that combine arithmetic calculations with relational comparisons.

Tools/Software Requirement:

- The C++ IDE installed and configured in earlier labs (for example, Microsoft Visual Studio or the IDE specified by your instructor).
- A working C++ compiler integrated with the IDE.
- MS Word or equivalent for preparing the lab report.
- An online C++ compiler may be used only as an instructor-directed fallback or quick comparison; the main work should be demonstrated in the installed IDE.

03-Hour Lab Plan:

Briefing and operator recap: 10 minutes | Tasks 1-5: 150 minutes | Report completion and submission check: 20 minutes.

Task 1 [CLO 1] - Arithmetic Operator Explorer (25 minutes):

Create a C++ program that reads two integers and displays the result of each arithmetic operator. This task is a short warm-up, but you must predict the results before running the program.

```
#include <iostream>

int main() {
    int firstNumber, secondNumber;
    std::cout << "Enter two integers (second must be non-zero): ";
    std::cin >> firstNumber >> secondNumber;

    std::cout << "Sum = " << firstNumber + secondNumber << "\n";
    std::cout << "Difference = " << firstNumber - secondNumber << "\n";
    std::cout << "Product = " << firstNumber * secondNumber << "\n";
    std::cout << "Integer quotient = " << firstNumber / secondNumber << "\n";
    std::cout << "Remainder = " << firstNumber % secondNumber << "\n";
    return 0;
}
```

1. Before running the code, predict all five results for firstNumber = 17 and secondNumber = 5.
2. Run the program with (17, 5), compare the actual output with your prediction, and correct any misunderstanding.
3. Run a second test with (20, 4). Use only a non-zero second integer because division by zero is not valid.
4. In your report, explain in 2-3 sentences why 17 / 5 gives 3 while 17 % 5 gives 2 when both operands are int.

Deliverable: C++ code, a prediction table for (17, 5), screenshots of both test runs, and the short explanation of / and %.

Task 2 [CLO 1] - Time Decomposer Using Division and Remainder (30 minutes):

Develop a program that converts a total number of seconds into hours, minutes, and seconds. The purpose is to use integer division and remainder in a practical calculation.

1. Declare totalSeconds, hours, remainingSeconds, minutes, and seconds as int. Read totalSeconds using `std::cin`.
2. Calculate hours using `totalSeconds / 3600`.
3. Calculate remainingSeconds using `totalSeconds % 3600`.
4. Calculate minutes from `remainingSeconds / 60` and seconds from `remainingSeconds % 60`.
5. Display the result clearly in the form: Hours: ..., Minutes: ..., Seconds: Add at least two useful comments.
6. Test the program with 3672 seconds and 7325 seconds. Verify each result manually before taking screenshots.

Expected result check for the required tests:

```
3672 seconds -> 1 hour(s), 1 minute(s), 12 second(s)
7325 seconds -> 2 hour(s), 2 minute(s), 5 second(s)
```

Deliverable: C++ code, manual calculations, and screenshots of both test runs.

Task 3 [CLO 1] - Relational Operator Comparison Console (30 minutes):

Build a small comparison program that reads two integers and evaluates all six relational operators. Remember that C++ normally prints a Boolean result as 1 for true and 0 for false.

Part A - Predict first: For the pair (18, 12), predict the result of `==`, `!=`, `>`, `<`, `>=`, and `<=` before writing or running the program.

Part B - Build the comparator: Read two integers, store each comparison in a bool variable, and print all six results with clear labels.

Example output layout:

```
first == second : 0
first != second : 1
first > second : 1
first < second : 0
first >= second : 1
first <= second : 0
```

1. Use one bool variable for each relational expression so that the data type of the comparison result is visible in your code.
2. Test the program with (18, 12) and then with equal values (15, 15).
3. In your report, explain why the second test changes the results of `==`, `!=`, `>=`, and `<=` compared with an unequal pair.

Deliverable: Prediction table, final C++ code, screenshots of both test runs, and the short explanation of the equal-value case.

Task 4 [CLO 3] - Compiler Detective: Arithmetic and Relational Errors (30 minutes):

Work through the following three mini-debugging challenges. Compile each version, read the first useful compiler message or observe the wrong output, then fix the operator-related problem. Record only the important message or observation in your report.

Challenge A - Arithmetic logic error

```
#include <iostream>
int main() {
    int first = 10, second = 20;
    double average = first + second / 2.0;
    std::cout << "Average = " << average << "\n";
    return 0;
}
```

Challenge B - Remainder operator type error

```
#include <iostream>
int main() {
    double totalItems = 17.0, groupSize = 4.0;
    std::cout << totalItems % groupSize << "\n";
    return 0;
}
```

Challenge C - Relational operator spelling errors

```
#include <iostream>
int main() {
    int scoreA = 60, scoreB = 75;
    std::cout << (scoreA => scoreB) << "\n";
    std::cout << (scoreA =< scoreB) << "\n";
    return 0;
}
```

1. For Challenge A, change the arithmetic expression so the average of 10 and 20 is 15. Explain why the original expression gives the wrong result.
2. For Challenge B, explain why % cannot be used with double operands. Correct the program by using suitable integer values for a remainder calculation.
3. For Challenge C, correct the two relational operators and run the program. Write the corrected operators exactly as C++ expects them.
4. Classify each problem as either a compile-time/operator error or a logic error, and include one relevant IDE error/build screenshot.

Deliverable: Corrected code for Challenges A-C, one IDE error/build screenshot, one successful execution screenshot, and your error classifications.

Task 5 [CLO 3] - Build from a Specification: Shopping Bill and Budget Checker (35 minutes):

Create this program from a blank source file. It combines arithmetic calculations with relational comparisons without using if/else or logical operators.

1. Before coding, write 8-10 lines of pseudocode using START, INPUT, CALCULATE, COMPARE, OUTPUT, and END.
2. Read item1Price and item2Price as double, item1Quantity and item2Quantity as int, and budget as double. Use positive quantities.
3. Calculate item1Cost, item2Cost, totalCost, totalItems, averageCostPerItem, and remainingBudget using only arithmetic expressions.
4. Display the calculated values, then display these comparisons with labels: totalCost <= budget, totalCost > budget, item1Cost == item2Cost, item1Price != item2Price, and item1Quantity >= item2Quantity.
5. Test Case 1: item1Price=120.5, item1Quantity=2, item2Price=75, item2Quantity=3, budget=500. Test Case 2: item1Price=50, item1Quantity=3, item2Price=50, item2Quantity=3, budget=250.
6. In your report, state which printed values are arithmetic results and which printed values are relational Boolean results (1=true, 0=false).

Deliverable: C++ code, screenshots of both test runs, and the requested arithmetic-vs-relational explanation.

Deliverables:

Compile a single Word document and submit it on LMS. Organize the report under Task 1 through Task 5. For each task, include the requested code, prediction/manual work where applicable, output/IDE screenshot(s), and short explanation or observation. The submitted code must be the code demonstrated in the lab.

Lab Rubrics

Your Lab 3 will be graded according to the following rubrics.

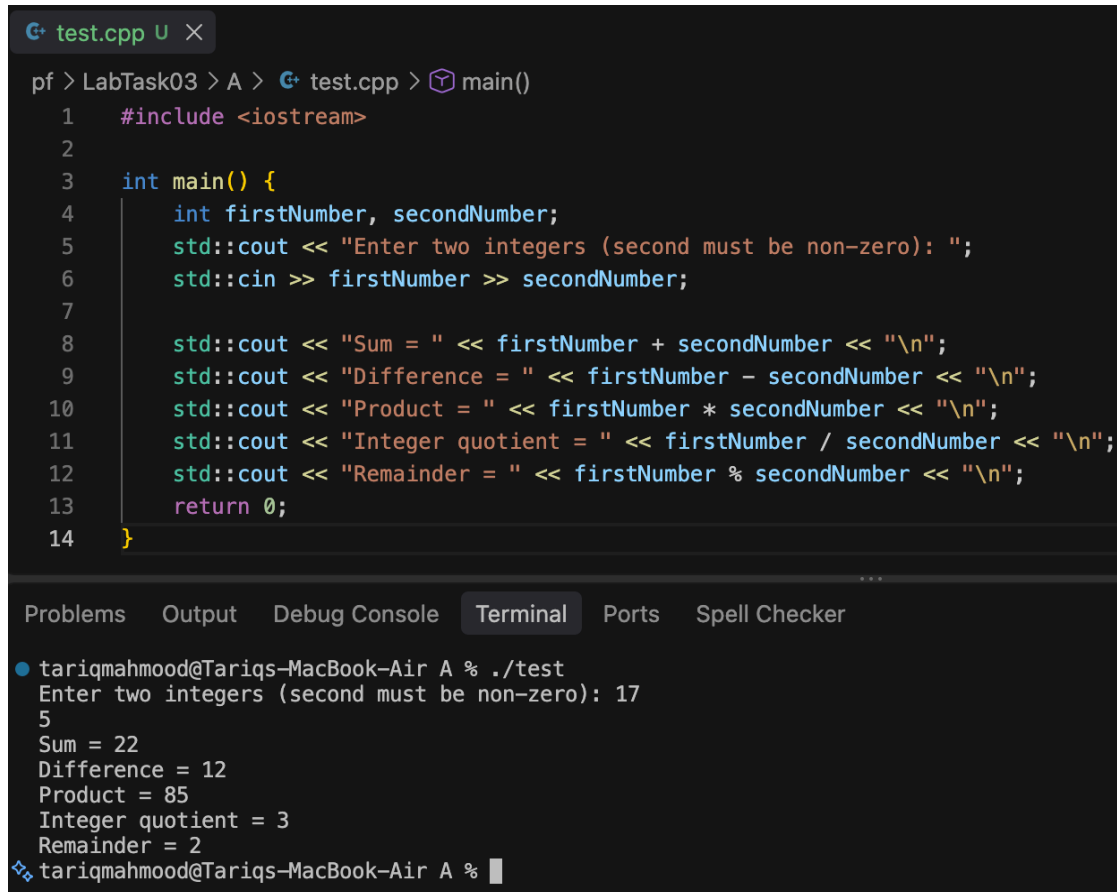
Lab Rubrics for Lab 3					
Sr. No.	Assessment	Unacceptable (0 Marks)	Does Not Meet Expectations (1/2 Marks)	Meets Expectations (3/4 Marks)	Exceeds Expectations (5 Marks)
1	Arithmetic and relational operator use (CLO1, PLO1)	The student did not submit the required work, submitted work that cannot be evaluated, or used unfair means/plagiarized work.	The student is unable to use arithmetic and relational operators correctly. Code contains major operator or expression errors and cannot be completed without substantial help.	The student demonstrates partial understanding of arithmetic calculations, integer division/remainder, and relational comparisons. Most tasks are attempted and code can be made to run with limited guidance, although some mistakes remain.	The student correctly uses +, -, *, /, %, ==, !=, >, <, >=, and <=; explains integer division and remainder; interprets Boolean 1/0 correctly; and writes readable, independently produced code.
2	IDE usage, testing/debugging, and documentation (CLO3, PLO5)		The student has difficulty creating/building/running programs in the IDE, cannot use compiler messages or output observations to correct basic operator errors, and provides incomplete documentation.	The student can create, build, run, and test the programs and can fix common arithmetic/relational errors with limited help. Screenshots, comments, and explanations are mostly complete and organized.	The student confidently compiles, runs, tests, and debugs the programs; uses evidence from compiler messages and output; and submits complete, well-structured code, screenshots, predictions, and explanations.

Task1

**I think the outputs would be as follows for
FirstNumber = 17 and SecondNumber = 5:**

- Sum = 22 (17 + 5)
- Difference = 12 (17 - 5)
- Product = 85 (17 x 5)
- Integer Quotient = 3 (17 / 5)
- Remainder = 2 (17 / 5 = 3 , remainder 2)

Actual:



```
test.cpp U X
pf > LabTask03 > A > test.cpp > main()
1  #include <iostream>
2
3  int main() {
4      int firstNumber, secondNumber;
5      std::cout << "Enter two integers (second must be non-zero): ";
6      std::cin >> firstNumber >> secondNumber;
7
8      std::cout << "Sum = " << firstNumber + secondNumber << "\n";
9      std::cout << "Difference = " << firstNumber - secondNumber << "\n";
10     std::cout << "Product = " << firstNumber * secondNumber << "\n";
11     std::cout << "Integer quotient = " << firstNumber / secondNumber << "\n";
12     std::cout << "Remainder = " << firstNumber % secondNumber << "\n";
13     return 0;
14 }
```

Problems Output Debug Console Terminal Ports Spell Checker

```
tariqmahmood@Tariqs-MacBook-Air A % ./test
Enter two integers (second must be non-zero): 17
5
Sum = 22
Difference = 12
Product = 85
Integer quotient = 3
Remainder = 2
tariqmahmood@Tariqs-MacBook-Air A %
```

Test with (20 , 4):

```
tariqmahmood@Tariqs-MacBook-Air A % ./test
Enter two integers (second must be non-zero): 20
4
Sum = 24
Difference = 16
Product = 80
Integer quotient = 5
Remainder = 0
❖ tariqmahmood@Tariqs-MacBook-Air A %
```

When we do $17 / 5$, it performs integer division giving us ONLY the integer part of the quotient of $17 / 5$ which is 3.

However, $17 \% 5$ gives us the remainder for the calculation $17 / 5$. Since 17 is not fully divisible by 5, it leaves a remainder (that being 2)

Task2

```

C: B.cpp U X
pf > LabTask03 > B > C: B.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int totalSeconds, hours, remainingSeconds, minutes, seconds;
7      cout << "Enter Total Seconds: ";
8      cin >> totalSeconds;
9      hours = totalSeconds / 3600;
10     remainingSeconds = totalSeconds % 3600;
11     minutes = remainingSeconds / 60;
12     seconds = remainingSeconds % 60;
13
14     cout << totalSeconds << " seconds -> " << hours << " hour(s) " << minutes << " minute(s) " << seconds << " Second(s)";
15
16     return 0;
17 }

```

```

Problems  Output  Debug Console  Terminal  Ports  Spell Checker
❖ tariqmahmood@Tariqs-MacBook-Air B % ./B
Enter Total Seconds: 3672
3672 seconds -> 1 hour(s) 1 minute(s) 12 Second(s)
❖ tariqmahmood@Tariqs-MacBook-Air B % ./B
Enter Total Seconds: 7325
7325 seconds -> 2 hour(s) 2 minute(s) 5 Second(s)
❖ tariqmahmood@Tariqs-MacBook-Air B %

```

3672 can be split into 3600 + 60 +12 seconds

So thats 1 hour , 1 min(s), 12 seconds

7325 can be split into 7200 + 120 + 5

Which is 2 hours , 2 minutes, 5 seconds

Task3

Logical Statement	Prediction
$18 == 12$	0
$18 != 12$	1
$18 > 12$	1
$18 < 12$	0
$18 > = 12$	1
$18 < = 12$	0

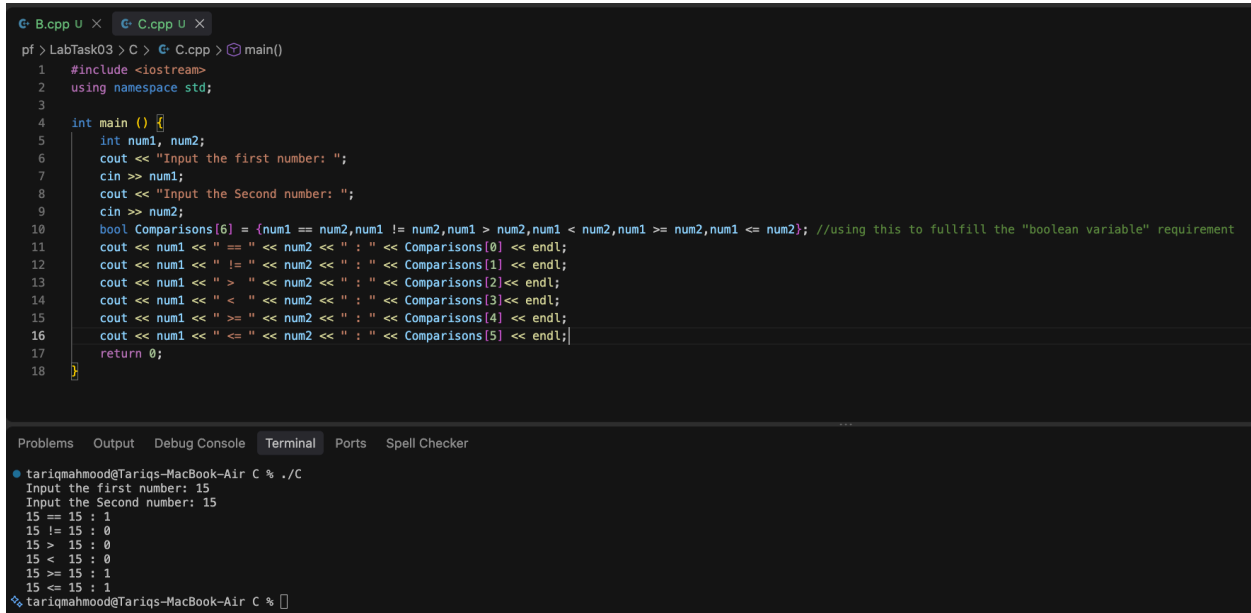
Actual (test with (18,12)):

```
G: B.cpp U X  C.cpp U X
pf > LabTask03 > C > G: C.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main () {
5      int num1, num2;
6      cout << "Input the first number: ";
7      cin >> num1;
8      cout << "Input the Second number: ";
9      cin >> num2;
10     bool Comparisons[6] = {num1 == num2, num1 != num2, num1 > num2, num1 < num2, num1 >= num2, num1 <= num2}; //using this to fullfill the "boolean variable" requirement
11     cout << num1 << " == " << num2 << " : " << Comparisons[0] << endl;
12     cout << num1 << " != " << num2 << " : " << Comparisons[1] << endl;
13     cout << num1 << " > " << num2 << " : " << Comparisons[2] << endl;
14     cout << num1 << " < " << num2 << " : " << Comparisons[3] << endl;
15     cout << num1 << " >= " << num2 << " : " << Comparisons[4] << endl;
16     cout << num1 << " <= " << num2 << " : " << Comparisons[5] << endl;
17     return 0;
18 }

Problems  Output  Debug Console  Terminal  Ports  Spell Checker
tarigmahmood@Tariqs-MacBook-Air B % ./B
Enter Total Seconds: 3672
3672 seconds -> 1 hour(s) 1 minute(s) 12 Second(s)
tarigmahmood@Tariqs-MacBook-Air B % ./B
Enter Total Seconds: 7325
7325 seconds -> 2 hour(s) 2 minute(s) 5 Second(s)
tarigmahmood@Tariqs-MacBook-Air B % clang++ C.cpp -o C
clang++: error: no such file or directory: 'C.cpp'
clang++: error: no input files
tarigmahmood@Tariqs-MacBook-Air B % cd ..
tarigmahmood@Tariqs-MacBook-Air LabTask03 % cd C
tarigmahmood@Tariqs-MacBook-Air C % clang++ C.cpp -o C
tarigmahmood@Tariqs-MacBook-Air C % ./C
Input the first number: 18
Input the Second number: 12
18 == 12 : 0
18 != 12 : 1
18 > 12 : 1
18 < 12 : 0
18 >= 12 : 1
18 <= 12 : 0
tarigmahmood@Tariqs-MacBook-Air C %
```

As you can see, I went with an array based approach and stored each comparison in a boolean array to fulfill the “ Use one bool variable for each relational expression” requirement.

test with (15,15)



```
1 #include <iostream>
2 using namespace std;
3
4 int main () {
5     int num1, num2;
6     cout << "Input the first number: ";
7     cin >> num1;
8     cout << "Input the Second number: ";
9     cin >> num2;
10    bool Comparisons[6] = {num1 == num2, num1 != num2, num1 > num2, num1 < num2, num1 >= num2, num1 <= num2}; //using this to fullfill the "boolean variable" requirement
11    cout << num1 << " == " << num2 << " : " << Comparisons[0] << endl;
12    cout << num1 << " != " << num2 << " : " << Comparisons[1] << endl;
13    cout << num1 << " > " << num2 << " : " << Comparisons[2] << endl;
14    cout << num1 << " < " << num2 << " : " << Comparisons[3] << endl;
15    cout << num1 << " >= " << num2 << " : " << Comparisons[4] << endl;
16    cout << num1 << " <= " << num2 << " : " << Comparisons[5] << endl;
17    return 0;
18 }
```

Problems Output Debug Console Terminal Ports Spell Checker

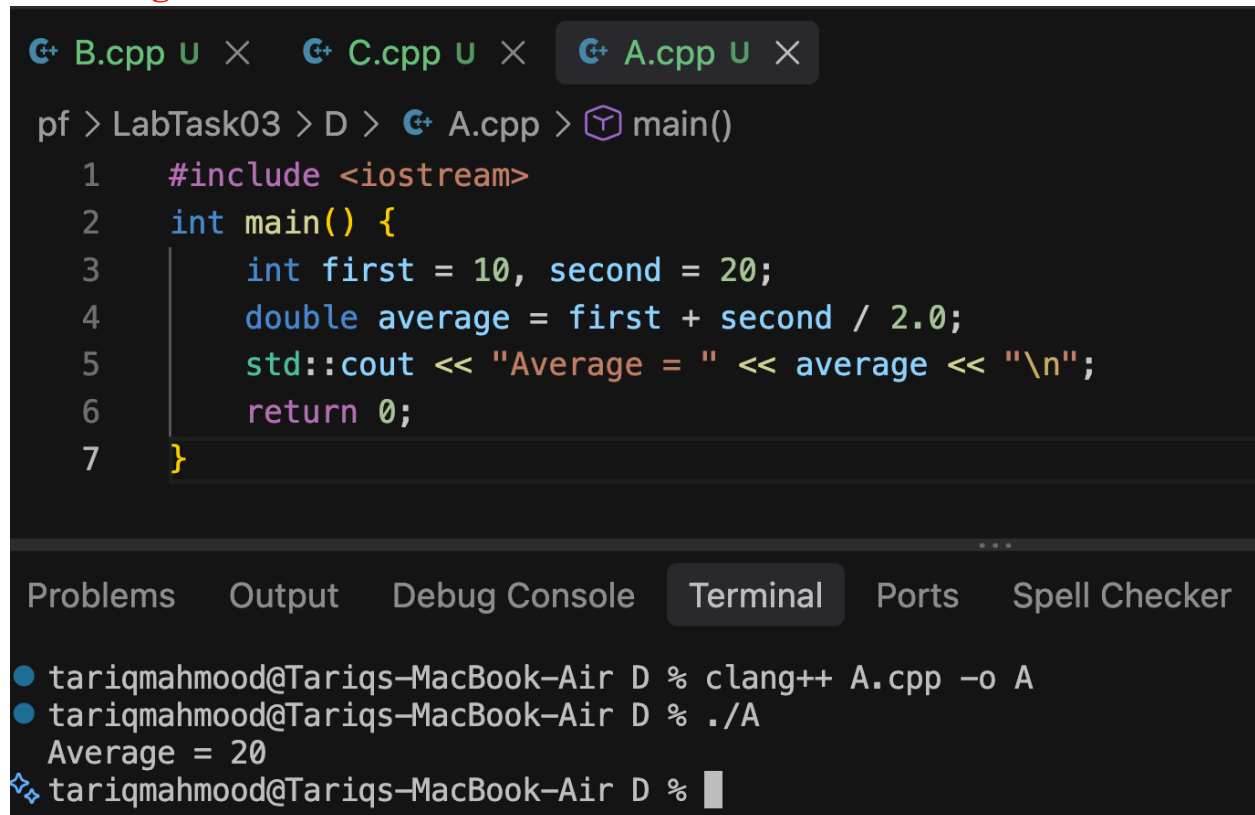
```
tarigmahmood@Tariqs-MacBook-Air C % ./C
Input the first number: 15
Input the Second number: 15
15 == 15 : 1
15 != 15 : 0
15 > 15 : 0
15 < 15 : 0
15 >= 15 : 1
15 <= 15 : 1
tarigmahmood@Tariqs-MacBook-Air C %
```

As you can see, $15 == 15$ evaluates to true since both values are equal. Similarly $15 > 15$ and $15 < 15$ both evaluate to false since the numbers are EQUAL not lesser then or greater then each other. However, the results for $15 \geq 15$ and $15 \leq 15$ still evaluate to true , the reason being the “=” in the operator which includes the number 15 in both “greater then **equal** to” and “lesser then **equal** to”.

Task4

Problem A:

PreDebug:



```

G+ B.cpp U ×  G+ C.cpp U ×  G+ A.cpp U ×
pf > LabTask03 > D > G+ A.cpp > main()
1  #include <iostream>
2  int main() {
3      int first = 10, second = 20;
4      double average = first + second / 2.0;
5      std::cout << "Average = " << average << "\n";
6      return 0;
7  }

Problems  Output  Debug Console  Terminal  Ports  Spell Checker

● tariqmahmood@Tariqs-MacBook-Air D % clang++ A.cpp -o A
● tariqmahmood@Tariqs-MacBook-Air D % ./A
Average = 20
❖ tariqmahmood@Tariqs-MacBook-Air D %
```

Explanation of Error:

Line 4, should be $(\text{first} + \text{second}) / 2$, currently the compiler reads this as $\text{first} + (\text{second}/2)$ reading to the wrong answer.

Classification: Logical Error

PostDebug:

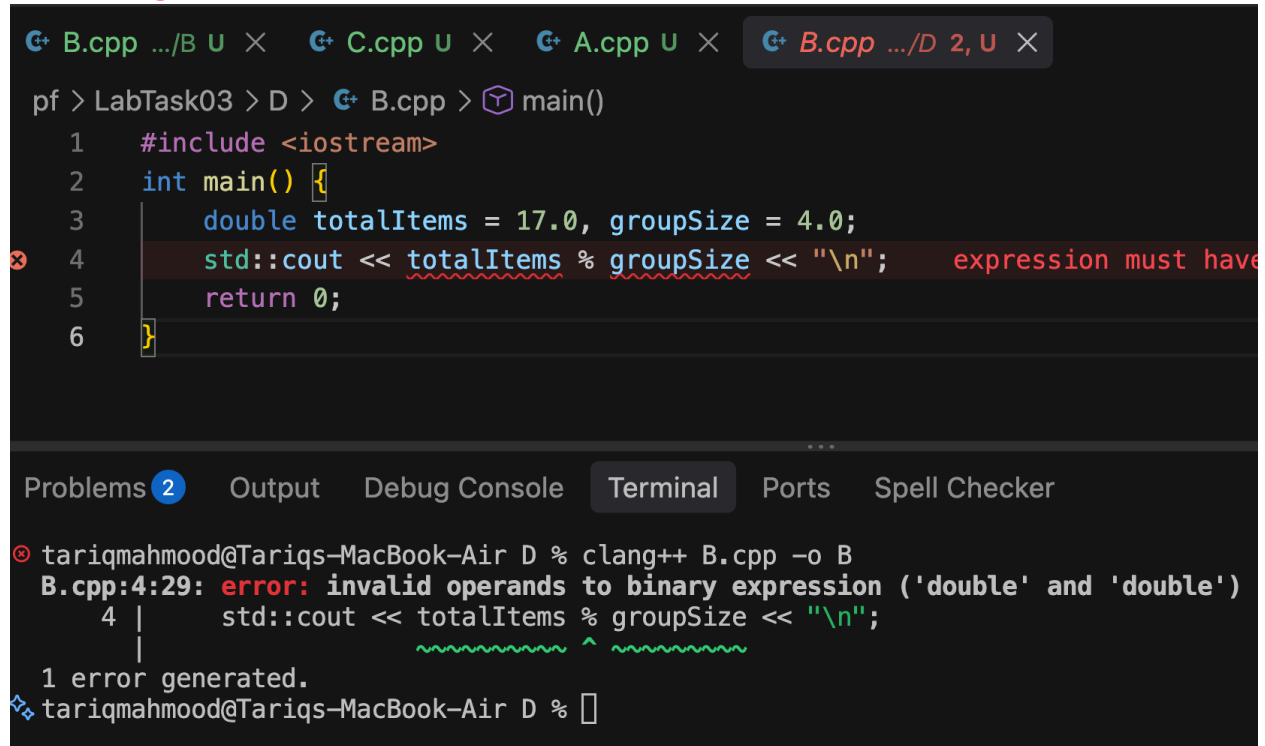
```
G+ B.cpp U × G+ C.cpp U × G+ A.cpp U ×
pf > LabTask03 > D > G+ A.cpp > Y main()
1  #include <iostream>
2  int main() {
3      int first = 10, second = 20;
4      double average = (first + second) / 2.0;
5      std::cout << "Average = " << average << "\n";
6      return 0;
7  }
```

Problems Output Debug Console Terminal Ports Spell C

- tariqmahmood@Tariqs-MacBook-Air D % clang++ A.cpp -o A
- tariqmahmood@Tariqs-MacBook-Air D % ./A
Average = 20
- tariqmahmood@Tariqs-MacBook-Air D % clang++ A.cpp -o A
- tariqmahmood@Tariqs-MacBook-Air D % ./A
Average = 15
- ✧ tariqmahmood@Tariqs-MacBook-Air D % █

Problem B:

PreDebug:



```
pf > LabTask03 > D > G+ B.cpp > main()
1  #include <iostream>
2  int main() {
3      double totalItems = 17.0, groupSize = 4.0;
4      std::cout << totalItems % groupSize << "\n";
5      return 0;
6  }
```

Problems 2 Output Debug Console Terminal Ports Spell Checker

```
❶ tariqmahmood@Tariqs-MacBook-Air D % clang++ B.cpp -o B
B.cpp:4:29: error: invalid operands to binary expression ('double' and 'double')
    4 |         std::cout << totalItems % groupSize << "\n";
      |                               ~~~~~^~~~~~
1 error generated.
❷ tariqmahmood@Tariqs-MacBook-Air D %
```

Explanation of Error:

Line 3 and Line 4. Variables are declared as integers on line 3 but on line 4, the program tries to calculate the integer of 2 double/floating point numbers which is not allowed in standard c++.

Classification: Compile Time Error

PostDebug:

```

G+ B.cpp .../B U ×  G+ C.cpp U ×  G+ A.cpp U ×  G+ B.cpp .../D U ×
pf > LabTask03 > D > G+ B.cpp > main()
1  #include <iostream>
2  int main() {
3      int totalItems = 17, groupSize = 4;
4      std::cout << totalItems % groupSize << "\n";
5      return 0;
6  }

Problems  Output  Debug Console  Terminal  Ports  Spell Checker

❶ tariqmahmood@Tariqs-MacBook-Air D % clang++ B.cpp -o B
B.cpp:4:29: error: invalid operands to binary expression ('double' and 'double')
4 |         std::cout << totalItems % groupSize << "\n";
  |                             ~~~~~^~~~~~
1 error generated.
❷ tariqmahmood@Tariqs-MacBook-Air D % clang++ B.cpp -o B
❸ tariqmahmood@Tariqs-MacBook-Air D % ./B
1
❹ tariqmahmood@Tariqs-MacBook-Air D %
```

Problem C:

PreDebug:

```
.../B U X C++ C.cpp .../C U X C++ A.cpp U X C++ B.cpp .../D U X C++ C.cpp .../E  
pf > LabTask03 > D > C++ C.cpp > main()  
1 #include <iostream>  
2 int main() {  
3     int scoreA = 60, scoreB = 75;  
4     std::cout << (scoreA => scoreB) << "\n";  
5     std::cout << (scoreA == scoreB) << "\n";  
6     return 0;  
7 }  
8
```

Problems 2 Output Debug Console Terminal Ports Spell Checker

```
@ tariqmahmood@Tariqs-MacBook-Air D % clang++ C.cpp -o C  
C.cpp:4:27: error: expected expression  
4 |         std::cout << (scoreA => scoreB) << "\n";  
    |                             ^  
C.cpp:5:27: error: expected expression  
5 |         std::cout << (scoreA == scoreB) << "\n";  
    |                             ^  
2 errors generated.  
tariqmahmood@Tariqs-MacBook-Air D %
```

Explanation of Error:

Syntax error in typing the error on lines 4 and 5

Classification: Operator(Syntax) error

PostDebug:

```
pp .../B U × C++ C.cpp .../C U × C++ A.cpp U × C++ B.cpp .../
pf > LabTask03 > D > C++ C.cpp > main()
1  #include <iostream>
2  int main() {
3      int scoreA = 60, scoreB = 75;
4      std::cout << (scoreA >= scoreB) << "\n";
5      std::cout << (scoreA <= scoreB) << "\n";
6      return 0;
7  }
8

Problems Output Debug Console Terminal Ports Spe
❌ tariqmahmood@Tariqs-MacBook-Air D % clang++ C.cpp -o C
C.cpp:4:27: error: expected expression
4 |     std::cout << (scoreA >= scoreB) << "\n";
  |                               ^
C.cpp:5:27: error: expected expression
5 |     std::cout << (scoreA <= scoreB) << "\n";
  |                               ^
2 errors generated.
● tariqmahmood@Tariqs-MacBook-Air D % clang++ C.cpp -o C
● tariqmahmood@Tariqs-MacBook-Air D % ./C
0
1
❖ tariqmahmood@Tariqs-MacBook-Air D %
```

Task5

Pseudocode:

START

DECLARE item1Price,item2Price AS DOUBLE

DECLARE item1Quantity , item2Quantity AS INTEGER

DECLARE Budget AS DOUBLE

INPUT item1Price

INPUT item2Price

INPUT item1Quantity

INPUT item2Quantity

INPUT budget

CALCULATE item1Cost (qty * price)

CALCULATE item2Cost (qty * price)

CALCULATE Total Cost as Sum of item1 and item2 Costs

CALCULATE Total Items as sum of item1 and item 2 quantities

CALCULATE Average Cost Per Item as TotalCost / TotalItems

CALCULATE Remaining Budget as Budget - TotalCost

OUTPUT Calculated Values

OUTPUT logical Comparisons

END

TestCase1:

```
Last login: Wed Sep 16 09:39:17 on ttys000
[tariqmahmood@Tariqs-MacBook-Air ~ % cd /Users/tariqmahmood/Projects/nust/pf/LabTask03
[tariqmahmood@Tariqs-MacBook-Air LabTask03 % clang++ bill.cpp -o ./bill
clang++: error: no such file or directory: 'bill.cpp'
clang++: error: no input files
[tariqmahmood@Tariqs-MacBook-Air LabTask03 % cd E
[tariqmahmood@Tariqs-MacBook-Air E % clang++ bill.cpp -o ./bill
[tariqmahmood@Tariqs-MacBook-Air E % ./bill
Enter Item 1 Price: 120.5
Enter Item1 Quantity: 2
Enter Item 2 Price: 75
Enter Item 2 Quantity: 3
Enter Budget: 500
item 1 Cost = 241
item 2 Cost = 225
Total Cost = 466
Total Items = 5
Average Cost per Item = 93.2
Remaininhg Budget = 34
totalCost <= budget : 1
totalCost > budget : 0
item1Cost == item2Cost : 0
item1Price != item2Price : 1
item1Quantity >= item2Quantity : 0
[tariqmahmood@Tariqs-MacBook-Air E % █
```

TestCase2:

```
[tariqmahmood@Tariqs-MacBook-Air E % ./Bill
Enter Item 1 Price: 50
Enter Item1 Quantity: 3
Enter Item 2 Price: 50
Enter Item 2 Quantity: 3
Enter Budget: 250
item 1 Cost = 150
item 2 Cost = 150
Total Cost = 300
Total Items = 6
Average Cost per Item = 50
Remaininh Budget = -50
totalCost <= budget : 0
totalCost > budget : 1
item1Cost == item2Cost : 1
item1Price != item2Price : 0
item1Quantity >= item2Quantity : 1
tariqmahmood@Tariqs-MacBook-Air E %
```

The Calculated Values (item1/2 cost/quantity ,budget , average cost per item etc)are all arithmetic results. All outputs with a comparison operator are logical outputs/results (<=, >= != , == etc).

C++ Code:

```
G- B.cpp .../B U X G- C.cpp .../C U X G- A.cpp U X G- B.cpp .../D U X G- C.cpp .../D U X G- bill.cpp U X
pf > LabTask03 > E > G- bill.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      double item1Price, item2Price, budget;
7      int item1Quantity, item2Quantity;
8
9      cout << "Enter Item 1 Price: ";
10     cin >> item1Price;
11     cout << "Enter Item1 Quantity: ";
12     cin >> item1Quantity;
13
14     cout << "Enter Item 2 Price: ";
15     cin >> item2Price;
16     cout << "Enter Item 2 Quantity: ";
17     cin >> item2Quantity;
18
19     cout << "Enter Budget: ";
20     cin >> budget;
21     double item1Cost = item1Price * item1Quantity;
22     double item2Cost = item2Price * item2Quantity;
23     double totalCost = item1Cost + item2Cost;
24     int totalItems = item1Quantity + item2Quantity;
25     double averageCostPerItem = totalCost / totalItems;
26     double remainingBudget = budget - totalCost;
27     cout << "item 1 Cost = " << item1Cost << endl;
28     cout << "item 2 Cost = " << item2Cost << endl;
29     cout << "Total Cost = " << totalCost << endl;
30     cout << "Total Items = " << totalItems << endl;
31     cout << "Average Cost per Item = " << averageCostPerItem << endl;
32     cout << "Remaininhg Budget = " << remainingBudget << endl;
33     cout << "totalCost <= budget : " << (totalCost <= budget) << endl; //using literals over here to keep code more concise
34     cout << "totalCost > budget : " << (totalCost > budget) << endl;
35     cout << "item1Cost == item2Cost : " << (item1Cost == item2Cost) << endl;
36     cout << "item1Price != item2Price : " << (item1Price != item2Price) << endl;
37     cout << "item1Quantity >= item2Quantity : " << (item1Quantity >= item2Quantity) << endl;
38
39
40
41
42     return 0;
43 }
```

Github Repo For all Code/Files:

<https://github.com/ARTariqDev/Nust-AI-2k26/tree/main/pf/LabTask03>

Submission By: Abdur Rehman Tariq, BSAI 2k26 CMSID: 623486